

INDIAN INSTITUTE OF TECHNOLOGY DELHI

GIPEDI Internship Programme

May – July 2026

Batch – 2

---

# Internship Report

Development of Embedded Systems

and

Design and Development of the Motion Data

Recorder System

---

Submitted by

**Arnab Roy**

**Internship Duration**      18 May 2026 – 13 July 2026

**Submission Date**      10 July 2026

# Acknowledgements

---

I would like to express my sincere gratitude to **Prof. Subrat Kar**, Department of Electrical Engineering, Indian Institute of Technology Delhi, for providing me with the opportunity to participate in the GIPEDI Internship Programme and for his guidance and encouragement throughout the internship.

I am especially thankful to my primary student mentor, **Chandan**, whose continuous technical guidance, constructive feedback, and willingness to share his knowledge greatly helped me throughout the development of the Motion Data Recorder project. I am also grateful to my secondary student mentor, **Utkarsh Roy**, for his valuable suggestions, technical support, and encouragement during different stages of the project.

I would also like to thank the coordinators and faculty members associated with the **GIPEDI Internship Programme** for organizing a well-structured internship that provided an excellent opportunity to gain practical experience in embedded systems, hardware design, firmware development, and software development.

I sincerely thank the **Indian Association for the Cultivation of Science (IACS)** for providing me with a strong academic foundation in Computer Science, which enabled me to successfully apply theoretical concepts to practical engineering problems during this internship.

Finally, I would like to thank my friends and peers for their support and encouragement throughout the internship. Their discussions, suggestions, and motivation made the learning experience both enjoyable and rewarding.

# Table of Contents

|                                                                         |            |
|-------------------------------------------------------------------------|------------|
| <b>Acknowledgements</b>                                                 | <b>i</b>   |
| <b>Table of Contents</b>                                                | <b>ii</b>  |
| <b>List of Tables</b>                                                   | <b>iii</b> |
| <b>List of Figures</b>                                                  | <b>iii</b> |
| <b>1 Introduction</b>                                                   | <b>iv</b>  |
| <b>2 Hardware and Software Resources</b>                                | <b>v</b>   |
| 2.1 Development Boards . . . . .                                        | v          |
| 2.2 Sensors . . . . .                                                   | v          |
| 2.3 Communication Interfaces . . . . .                                  | v          |
| 2.4 Storage Devices . . . . .                                           | vi         |
| 2.5 Display Modules . . . . .                                           | vi         |
| 2.6 Electronic Components . . . . .                                     | vi         |
| <b>3 Development Tools and Libraries</b>                                | <b>vii</b> |
| 3.1 Development Software . . . . .                                      | vii        |
| 3.2 Arduino Board Support Packages . . . . .                            | viii       |
| 3.3 Arduino Libraries . . . . .                                         | viii       |
| 3.3.1 Core Libraries . . . . .                                          | viii       |
| 3.3.2 Sensor Libraries . . . . .                                        | viii       |
| 3.3.3 RTC Libraries . . . . .                                           | viii       |
| 3.3.4 Display, Graphics & User Interface . . . . .                      | viii       |
| 3.3.5 Supporting Libraries . . . . .                                    | viii       |
| 3.4 Flutter Packages . . . . .                                          | ix         |
| 3.5 Bill of Materials (BOM) . . . . .                                   | ix         |
| <b>4 Work Completed During the Internship</b>                           | <b>x</b>   |
| 4.1 Simulation and Practical Application of Embedded Circuits . . . . . | x          |
| 4.2 Project of Data Recorder . . . . .                                  | x          |
| 4.2.1 System Architecture and Hardware Connections . . . . .            | x          |
| 4.2.2 Firmware and Data Logging . . . . .                               | xi         |
| 4.2.3 Bluetooth Low Energy (BLE) Integration . . . . .                  | xi         |

|          |                                                                |             |
|----------|----------------------------------------------------------------|-------------|
| 4.2.4    | Custom PCB Design . . . . .                                    | xi          |
| 4.2.5    | Procedure to Record and Transmit Data . . . . .                | xii         |
| <b>5</b> | <b>Challenges and Skills Developed</b>                         | <b>xvii</b> |
| 5.1      | Challenges Encountered . . . . .                               | xvii        |
| 5.2      | Skills Developed . . . . .                                     | xviii       |
| 5.2.1    | Arduino Development . . . . .                                  | xviii       |
| 5.2.2    | Embedded Systems Development (Seeed XIAO nRF52840 Sense) . . . | xviii       |
| 5.2.3    | Software Development . . . . .                                 | xviii       |
| 5.2.4    | Hardware Design . . . . .                                      | xviii       |
| <b>6</b> | <b>Conclusion</b>                                              | <b>xix</b>  |

## List of Tables

|     |                                                                     |     |
|-----|---------------------------------------------------------------------|-----|
| 3.1 | Development Software and Tools used during the Internship . . . . . | vii |
| 3.2 | Flutter Packages Used for Android Development . . . . .             | ix  |
| 3.3 | Bill of Materials for the Motion Data Recorder . . . . .            | ix  |
| 4.1 | Desktop Client Commands . . . . .                                   | xvi |

## List of Figures

|     |                                                                             |     |
|-----|-----------------------------------------------------------------------------|-----|
| 4.1 | System Architecture and Hardware Connections of the Data Recorder . . . . . | xi  |
| 4.2 | Custom PCB Design for the Data Recorder . . . . .                           | xii |
| 4.3 | Android Application Interface . . . . .                                     | xiv |
| 4.4 | Python Desktop Client . . . . .                                             | xvi |

# Chapter 1

## Introduction

---

Embedded systems have become an integral part of modern electronic devices, enabling intelligent sensing, data acquisition, automation, and wireless communication across numerous application domains. During the GIPEDI Internship Programme at the Indian Institute of Technology Delhi, I had the opportunity to gain practical experience in embedded systems through the design and implementation of several hardware and software projects.

The internship began with a series of laboratory assignments aimed at strengthening the fundamentals of embedded system design. These activities included digital input/output interfacing, display interfacing, temperature monitoring systems, scientific calculator development, waveform generation, RTC-based monitoring systems, and audio playback applications. These projects provided practical exposure to peripheral interfacing, communication protocols, firmware development, debugging, and embedded software design.

Building upon this foundation, the primary objective of the internship was the development of a portable **Bluetooth-enabled Motion Data Recorder**. The system was designed around the **Seeed XIAO nRF52840 Sense** development board and integrates motion sensing, real-time timestamp generation, data logging, wireless communication, and low-power operation into a compact embedded platform.

The developed system continuously acquires acceleration and angular velocity data from the onboard six-axis IMU sensor and stores timestamped measurements on a microSD card in CSV format using an external Real-Time Clock (RTC). To enhance usability, the Recorder communicates wirelessly with a custom Android application through Bluetooth Low Energy (BLE), allowing users to remotely control logging operations and manage stored files. A Python-based desktop client was also developed to provide an alternative interface for communication and testing.

In addition to firmware development, the project involved the design of a custom Printed Circuit Board (PCB), implementation of power management through deep sleep functionality, development of the companion Android application using Flutter, and preparation of comprehensive technical documentation describing the overall system.

This report presents the hardware platforms, software tools, technologies, libraries, and development methodologies employed throughout the internship. It also summarizes the completed work, discusses the implementation of the Motion Data Recorder project, highlights the challenges encountered during development, and outlines the technical knowledge and practical skills acquired during the internship.

# Chapter 2

## Hardware and Software Resources

---

This chapter provides an overview of the hardware platforms, electronic components, sensors, communication modules, storage devices, and display units used during the internship. These resources formed the foundation for the laboratory exercises as well as the development of the Bluetooth Motion Data Recorder.

### 2.1 Development Boards

The following embedded development boards were used throughout the internship for firmware development and hardware prototyping.

- Arduino Uno
- Espressif ESP32-WROOM-32
- Seeed XIAO nRF52840 Sense

### 2.2 Sensors

Various sensors were interfaced to acquire environmental and motion-related data.

- LSM6DS3 Six-Axis IMU (XIAO Inbuilt)
- LM35 Temperature Sensor

### 2.3 Communication Interfaces

The following communication technologies were employed for programming, peripheral interfacing, and wireless communication.

- Bluetooth Low Energy (BLE)
- USB (Micro USB and USB Type-C)
- I<sup>2</sup>C Communication Bus
- SPI Communication Bus
- UART [TX, RX]

## 2.4 Storage Devices

The Motion Data Recorder stores acquired sensor data on removable flash storage.

- microSD Card Module
- microSD Memory Card

## 2.5 Display Modules

Several display modules were utilized for user interaction and visualization during different laboratory tasks.

- Nokia PCD8544 LCD
- 16×2 Character LCD (JHD162A)
- LT543 Seven-Segment Display

## 2.6 Electronic Components

The following electronic components were used for hardware implementation and circuit development.

- Push Buttons
- LEDs
- Piezo Buzzer
- Relay Module
- L293 Motor Driver
- Potentiometers
- DS1302 RTC Module
- DS3231 RTC Module
- Breadboard
- Resistors
- Jumper Wires

# Chapter 3

## Development Tools and Libraries

This chapter describes the software tools, development environments, frameworks, board support packages, and software libraries used throughout the internship for firmware development, mobile application development, PCB design, documentation, and version control.

### 3.1 Development Software

Table 3.1: Development Software and Tools used during the Internship

| Software           | Version Details                                                              | Purpose                            |
|--------------------|------------------------------------------------------------------------------|------------------------------------|
| Arduino IDE        | Version: 2.3.10<br>Date: 2026-06-09                                          | Embedded firmware development      |
| Visual Studio Code | Version: 1.127.0 (Universal)<br>Node.js: 24.15.0<br>OS: Darwin arm64 (macOS) | Source code editing                |
| Flutter SDK        | Flutter 3.44.4 (stable channel)<br>Dart 3.12.2                               | Android app development            |
| Android Studio     | Quail 1   2026.1.1 Patch                                                     | Android build & debug              |
| Obsidian           | Version 1.12.7                                                               | Technical documentation            |
| KiCad              | Version: 10.0.3, release build<br>Platform: macOS Version 26.5.2             | PCB design                         |
| GitHub             | 2.54.0                                                                       | Source code management             |
| SimulIDE           | 1.1.0 SR2                                                                    | Simulation of Circuits             |
| LTSpice            | -                                                                            | Wiring Check                       |
| nRF Connect        | -                                                                            | Bluetooth connection to smartphone |

## 3.2 Arduino Board Support Packages

- **Arduino AVR Boards:** Arduino Uno development
- **ESP32 by Espressif Systems:** ESP32 firmware development
- **Seeed nRF52 Boards:** Seeed XIAO nRF52840 Sense support

## 3.3 Arduino Libraries

### 3.3.1 Core Libraries

- **Wire / SPI:** I<sup>2</sup>C and SPI communication
- **SD / SdFat:** microSD card file system and data logging
- **EEPROM:** Non-volatile data storage

### 3.3.2 Sensor Libraries

- **Seeed Arduino LSM6DS3:** 6-axis IMU (accelerometer and gyroscope)
- **DHT Sensor Library:** DHT11/DHT22 temperature and humidity sensors
- **OneWire & DallasTemperature:** DS18B20 digital temperature sensor
- **DS1621:** I<sup>2</sup>C temperature sensor

### 3.3.3 RTC Libraries

- **RTC / DS3231:** Real-Time Clock communication
- **Rtc by Makuna:** RTC interface supporting DS1302, DS3231 and related devices

### 3.3.4 Display, Graphics & User Interface

- **Adafruit GFX Library:** Graphics framework for displays
- **Adafruit PCD8544 Nokia 5110 LCD Library:** Nokia 5110 LCD interface
- **Adafruit SSD1306:** OLED display driver
- **Keypad:** Matrix keypad interface

### 3.3.5 Supporting Libraries

- **Adafruit BusIO:** Hardware communication abstraction for I<sup>2</sup>C and SPI devices
- **Adafruit Unified Sensor:** Standardized sensor interface
- **Adafruit SPIFlash:** SPI flash memory support

### 3.4 Flutter Packages

**Table 3.2:** Flutter Packages Used for Android Development

| Package            | Version | Purpose                                 |
|--------------------|---------|-----------------------------------------|
| flutter_blue_plus  | 2.3.10  | Bluetooth Low Energy communication      |
| permission_handler | 12.0.3  | Runtime permission management           |
| file_picker        | 10.3.10 | File selection from device storage      |
| shared_preferences | 2.5.3   | Local application settings              |
| url_launcher       | 6.3.1   | Launch external URLs and file locations |
| path_provider      | 2.1.5   | Access application storage directories  |
| cupertino_icons    | 1.0.8   | iOS-style icon set                      |

### 3.5 Bill of Materials (BOM)

The table below lists the major hardware components used in the development of the Motion Data Recorder.

**Table 3.3:** Bill of Materials for the Motion Data Recorder

| Sl. No. | Component                    | Quantity | Purpose                                    |
|---------|------------------------------|----------|--------------------------------------------|
| 1       | Seeed XIAO nRF52840 Sense    | 1        | Main microcontroller with onboard IMU      |
| 2       | DS3231 Real-Time Clock (RTC) | 1        | Accurate timestamp generation              |
| 3       | microSD Card Module & Card   | 1        | Storage interface and motion data storage  |
| 4       | Li-Po Battery (3.7 V)        | 1        | Portable power supply                      |
| 5       | Battery Mgmt. Circuit        | 1        | Battery charging and power management      |
| 6       | Push Button                  | 1        | User input (logging, BLE mode, deep sleep) |
| 7       | Custom PCB                   | 1        | Hardware integration                       |
| 8       | USB Type-C Connector         | 1        | Programming and battery charging           |

# Chapter 4

## Work Completed During the Internship

---

### 4.1 Simulation and Practical Application of Embedded Circuits

The following embedded system applications were designed and implemented during the initial phase of the internship:

- Digital Output and LED, Display Interfacing
- Temperature Monitoring and Weather Monitoring System
- Scientific Calculator
- Waveform Generator
- RTC-Based Monitoring System
- Audio Playback System
- Walkie-talkie (trans-reciever) using UART protocol

Each task focused on understanding different hardware interfaces, communication protocols, peripheral integration, and embedded programming techniques.

### 4.2 Project of Data Recorder

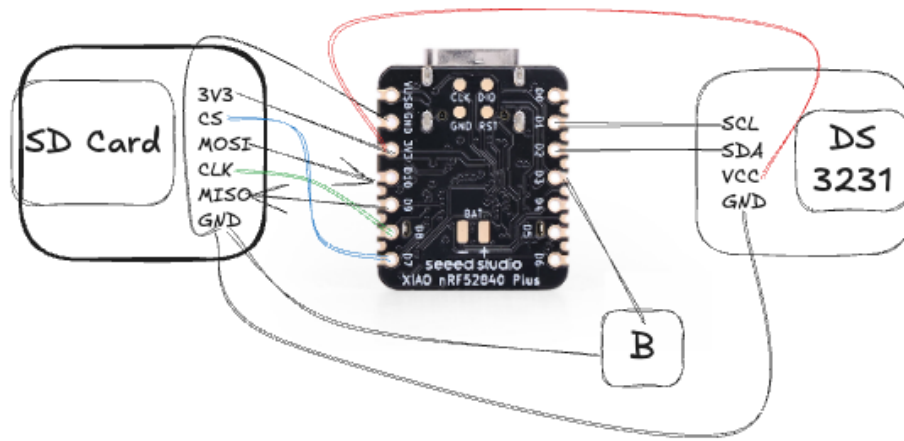
The primary project undertaken during the internship was the development of a Bluetooth-enabled Motion Data Recorder. The project involved designing a portable embedded system capable of acquiring motion data, storing timestamped measurements on a microSD card, communicating with a custom mobile application over Bluetooth Low Energy (BLE), and operating efficiently using deep sleep power management.

#### 4.2.1 System Architecture and Hardware Connections

The system is built around the **Seeed XIAO nRF52840 Sense**. It acts as the central controller, coordinating sensor acquisition from the onboard 6-axis IMU, timestamp generation, SD card storage, and wireless communication.

The hardware peripherals are mapped to the microcontrollers shown in Figure 4.1, the hardware peripherals are connected to the Seeed XIAO nRF52840 Sense as follows: as follows:

- **RTC (DS3231):** Interfaced via I<sup>2</sup>C (SCL to D1, SDA to D2, RST to D0).
- **microSD Card Module:** Interfaced via SPI (CS to D7, MOSI to D10, MISO to D9, SCK to D8).



**Figure 4.1:** System Architecture and Hardware Connections of the Data Recorder

- **User Interface:** A push button connected to D3 (with the other side to GND) for state control, alongside an LED connected to D0.

#### 4.2.2 Firmware and Data Logging

The firmware operates on an event-driven state machine, maintaining a fixed sampling interval of 100 ms (10 Hz) to balance storage efficiency and motion capture accuracy. The system logs the Date, Time, Millisecond counter, Acceleration (X, Y, Z axes), and Angular Velocity (X, Y, Z axes) in CSV format.

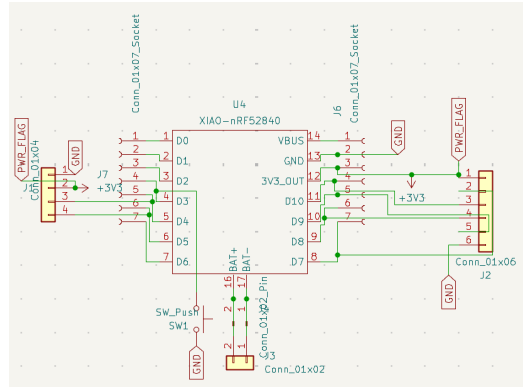
To ensure data integrity, the system automatically marks session start and stop points within the file and appends new sessions to existing daily logs rather than creating redundant files. The device utilizes a single push button to handle all user inputs: a single press toggles logging or wakes the device, a double press enables BLE, and a long press forces the system into deep sleep.

#### 4.2.3 Bluetooth Low Energy (BLE) Integration

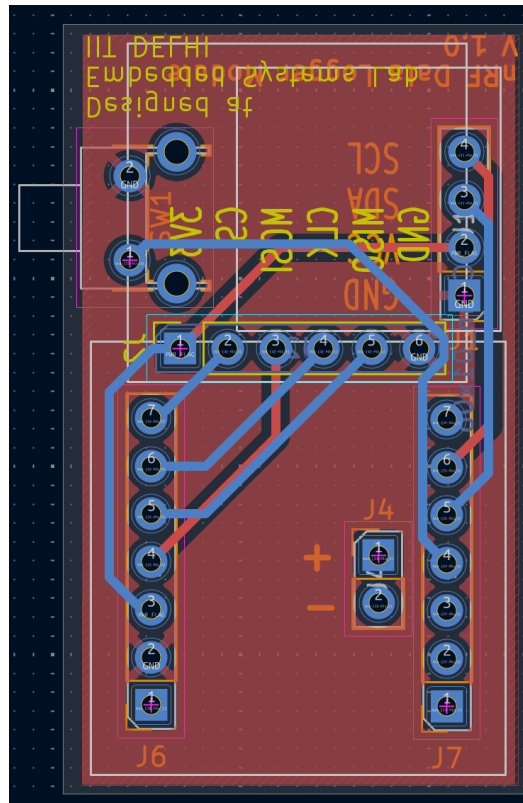
To minimize unnecessary power consumption, BLE (version 5.0) is enabled strictly on demand. Once activated, the Recorder advertises as "MotionLogger". The firmware exposes dedicated BLE characteristics that allow the Android app and a custom Python desktop client to communicate with the device. Supported remote commands include starting/stopping logs, checking system status, and wirelessly listing, downloading, or deleting stored CSV files without needing to physically remove the SD card.

#### 4.2.4 Custom PCB Design

To improve the portability and reliability of the Recorder, a custom Printed Circuit Board (PCB) was designed using KiCad. The PCB successfully replaced the initial breadboard prototype by providing a compact and organized hardware layout that integrates the Seesed XIAO, RTC module, microSD module and a 3.7V Li-Po battery. Figure 4.2a is the screenshot of the schematic and Figure 4.2b is the PCB design.



(a) Circuit Schematic



(b) PCB Layout

**Figure 4.2:** Custom PCB Design for the Data Recorder

#### 4.2.5 Procedure to Record and Transmit Data

The Motion Data Recorder is designed to operate through a single push button and communicates with external devices using Bluetooth Low Energy (BLE). The complete workflow begins with powering the device, followed by data acquisition, wireless communication, and finally low-power shutdown. Figure 4.3 illustrates the complete operational sequence.

#### Device Operation

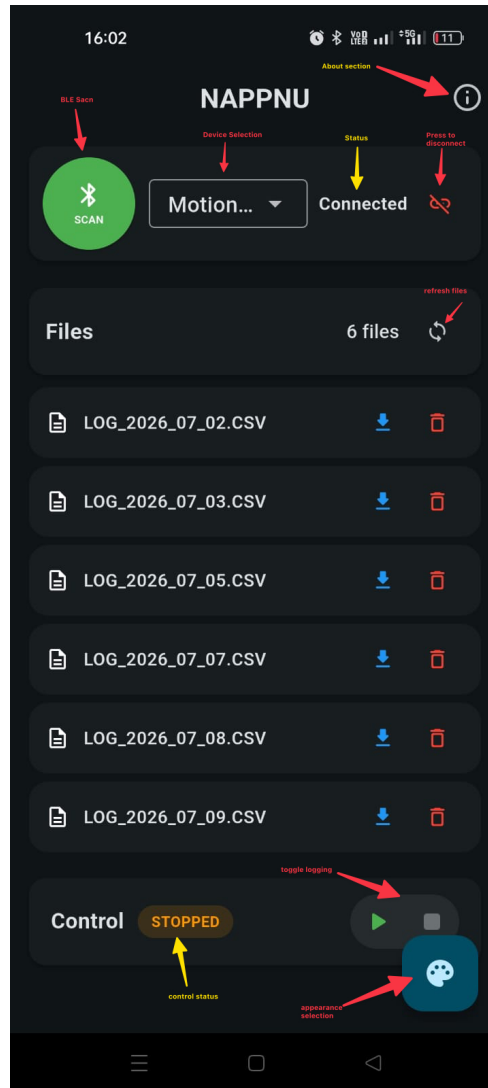
The recorder operates according to the following sequence.

1. Power the device using the **USB Type-C** connector or the **Li-Po battery**.
2. During **startup**, the firmware initialises the onboard IMU, RTC module, microSD card interface, Bluetooth stack, and internal peripherals.
3. After successful initialisation, the recorder enters the **Idle** state and waits for user input.
4. A **single button press** starts a new logging session. Sensor data is sampled at **10 Hz** and written to the daily CSV file stored on the microSD card.
5. During logging, the recorder continuously records:
  - Date and Time
  - Millisecond Counter
  - Accelerometer (X, Y, Z axes)
  - Gyroscope (X, Y, Z axes)
6. **Pressing the button once again** terminates the current logging session. The CSV file is flushed, updated, and safely closed.
7. A **double button press** enables Bluetooth advertising for **30 seconds**. During this period, the recorder becomes discoverable by nearby **Android devices** or the **desktop client**.
8. If a BLE connection is established within the advertising interval, the advertising stops automatically, and the recorder enters *Bluetooth communication mode*.
9. While connected, *remote commands* may be used to:
  - Start logging
  - Stop logging
  - Display recorder status
  - List of stored CSV files
  - Download selected files
  - Delete selected files
10. If no device connects within *30 seconds*, Bluetooth advertising automatically terminates, and the recorder returns to the Idle state.
11. A **long press (approximately 1.5 seconds)** places the recorder into **deep sleep mode** to reduce power consumption.
12. A **subsequent button press** wakes the recorder, reinitialises all peripherals, and returns it to the Idle state.

### Data Transmission to the Android Application

Before attempting to connect, the recorder must first enter Bluetooth advertising mode. From the idle state, press the hardware push button twice. The recorder advertises for approximately **30 seconds**. If a client connects during this period, advertising stops automatically, and the connection remains active until manually disconnected.

Figure 4.3 shows the user interface of the Android application.



**Figure 4.3:** Android Application Interface

The complete operating procedure is as follows.

1. Power on the Motion Data Recorder.
2. Press the hardware **push button twice** to enable BLE advertising.
3. Open the Android application.
4. Press the **Scan** button to search for nearby Motion Recorder devices.
5. Select the desired device from the **drop-down list**.

6. Wait until the connection status changes to **Connected**.
7. Press the *refresh button* to retrieve the latest list of CSV files stored on the microSD card.
8. To download a file, press the **blue download icon** beside the required file.
9. To permanently remove a file from the recorder, press the **delete icon** beside that file.
10. Use the **Play/Stop** control to start or stop motion data logging.
11. Observe the *logger state* indicator, which reports whether the recorder is currently Offline, Stopped or Logging.
12. Press the **Disconnect button** when all operations have been completed.

The controls available on the application interface perform the following functions.

- BLE Scan button
- Device selection drop-down
- Connection status indicator
- Disconnect button
- Refresh file list
- File download
- File deletion
- Logger status indicator
- Start/Stop logging control
- Theme selector
- About page

### Data Transmission to the Desktop Client

A lightweight Python desktop client was developed primarily for firmware testing and debugging. The client provides direct access to all BLE commands through a terminal interface.

The desktop client is operated using the following procedure.

1. Power on the Motion Data Recorder.
2. Press the hardware **push button twice** to start BLE advertising.
3. Launch the **Python desktop client**.
4. Wait while the client scans for nearby Motion Recorder devices.

```

~/.pr/G/ARDUINO/Exp/Task main !3 ?1 > source /.../.venv/bin/activate
~/.pr/G/ARDUINO/Exp/Task main !3 ?1 > /NAPPNU/connect.py
Scanning for MotionLogger...
Found: MotionLogger
Address: EDAA2CEF-4D0E-6029-C58B-9660E07736D0
Connected
Notifications enabled

Commands:
STATUS
START
STOP
LIST
G:<index>
D:<index>
EXIT

Command> 

```

Figure 4.4: Python Desktop Client

5. After the device is discovered, the client automatically establishes the BLE connection.
6. Once connected, the list of available commands is displayed in the terminal.
7. Enter the required command and press **Enter**.
8. Repeat commands as required.
9. Type **EXIT** to disconnect from the recorder and close the session.

The desktop client supports the following commands.

Table 4.1: Desktop Client Commands

| Command   | Description                                  |
|-----------|----------------------------------------------|
| STATUS    | Display the current logger status            |
| START     | Start motion data logging                    |
| STOP      | Stop motion data logging                     |
| LIST      | Display all CSV files stored on the recorder |
| G:<index> | Download the selected file                   |
| D:<index> | Delete the selected file                     |
| EXIT      | Disconnect and terminate the client          |

**Note:** The recorder supports only one active BLE connection. Therefore, either the Android application or the desktop client may be connected at a given time. Ensure that the current client is disconnected before attempting to connect using another interface.

# Chapter 5

## Challenges and Skills Developed

---

### 5.1 Challenges Encountered

The development of the Motion Data Recorder involved several practical challenges related to embedded system development, hardware integration, Bluetooth communication, and software development. Overcoming these challenges significantly improved my understanding of designing complete embedded systems.

- **Learning the Seeed XIAO nRF52840 Sense:** The project began with understanding the Seeed XIAO nRF52840 Sense, its onboard IMU, GPIOs, Bluetooth Low Energy capabilities, and development environment. Since the platform was new to me, considerable time was spent studying the official documentation and experimenting with sample programs before beginning the actual project.
- **Hardware Integration and Firmware Development:** After becoming familiar with the development board, the next challenge was integrating the onboard IMU, DS3231 Real-Time Clock, microSD card module, and push-button interface into a single embedded system. Each peripheral was individually tested before being integrated into the final firmware.
- **Device Control and State Management:** Developing an intuitive user interface using a single push button required implementing multiple operations such as starting and stopping data logging, enabling Bluetooth mode, and entering deep sleep. This was achieved by designing an event-driven state machine capable of reliably detecting single press, double press, and long press events.
- **Custom PCB Design:** Designing the custom PCB was one of the most time-consuming stages of the project. Since PCB design was completely new to me, I first learned schematic capture, footprint assignment, component placement, routing, and PCB design practices using KiCad. Multiple revisions and hardware verification were carried out before finalizing the design.
- **Hardware Debugging:** During hardware development, some prototype boards became unusable due to accidental short circuits and wiring mistakes while testing new features. Careful inspection of the hardware, verification of circuit connections, and more cautious debugging practices helped prevent similar issues in subsequent development.
- **Cross-Platform Software Development:** Once the embedded firmware was operational, companion applications were developed using Flutter for Android and Python for desktop systems.

- **Bluetooth Low Energy Communication and System Integration:** The final stage involved integrating the firmware with both the Android application and the desktop client through Bluetooth Low Energy. Developing custom BLE services and testing communication between all three platforms required repeated debugging and refinement to achieve reliable operation.

Successfully addressing these challenges strengthened my understanding of embedded firmware development, hardware–software integration, Bluetooth Low Energy communication, PCB design, and cross-platform application development.

## 5.2 Skills Developed

The internship provided hands-on experience in embedded systems, electronics, firmware development, PCB design, and software development. The skills acquired are categorized below.

### 5.2.1 Arduino Development

- Arduino IDE and Library Management, Embedded C/C++ Programming
- Digital Input/Output (GPIO) Interfacing, Analog-to-Digital Conversion (ADC), PWM Signal Generation
- I<sup>2</sup>C, SPI, and UART Communication
- Sensor Interfacing, Display Interfacing, SD Card File Handling, Real-Time Clock (RTC) Integration

### 5.2.2 Embedded Systems Development (Seeed XIAO nRF52840 Sense)

- Embedded Firmware Development
- Bluetooth Low Energy (BLE) Communication
- Motion Sensor (LSM6DS3) Integration
- Data Logging System Development
- Hardware Integration and System Testing

### 5.2.3 Software Development

- Android Application Development using Flutter
- Obsidian integration with various plugins such as Git, Auto-Journal etc.
- Technical Documentation using Markdown and L<sup>A</sup>T<sub>E</sub>X
- Python script writing

### 5.2.4 Hardware Design

- Schematic Design using KiCad
- PCB Layout Design
- Electronic Component Selection

# Chapter 6

## Conclusion

---

The GIPEDI Internship Programme at the Indian Institute of Technology Delhi provided valuable practical exposure to embedded systems design and development. Throughout the internship, I gained hands-on experience in firmware development, hardware interfacing, sensor integration, communication protocols, data acquisition, and embedded software debugging through the successful completion of various laboratory assignments and projects.

The development of the **Motion Data Recorder** served as the primary outcome of the internship, integrating the concepts learned throughout the programme into a complete embedded system. The project involved motion sensing, real-time timestamp generation, microSD-based data logging, Bluetooth Low Energy (BLE) communication, power management through deep sleep, Android application development using Flutter, and custom PCB design. Participating in every stage of the development process from system design and firmware implementation to hardware integration, mobile application development, testing, and documentation provided a comprehensive understanding of the complete embedded system development lifecycle.

The internship also strengthened my problem-solving and analytical skills by exposing me to real-world engineering challenges, including hardware debugging, BLE communication, reliable file management, PCB design constraints, and low-power optimization. Addressing these challenges enhanced both my technical competence and my confidence in designing and developing reliable embedded systems.

Overall, the internship successfully bridged the gap between theoretical knowledge and practical implementation. It significantly enhanced my understanding of embedded systems engineering and established a strong foundation for pursuing advanced academic research and professional opportunities in embedded systems, the Internet of Things (IoT), and related domains. The knowledge, technical skills, and practical experience gained during this internship will serve as valuable assets in my future academic and professional endeavors.

# References

---

- [1] Arduino, “Arduino Documentation.”. Available: <https://docs.arduino.cc/>.
- [2] Seeed Studio, “Seeed XIAO nRF52840 Sense Documentation.”. Available: [https://wiki.seeedstudio.com/XIAO\\_BLE/](https://wiki.seeedstudio.com/XIAO_BLE/).
- [3] Google, “Flutter Documentation.”. Available: <https://docs.flutter.dev/>.
- [4] KiCad Developers, “KiCad EDA Documentation.”. Available: <https://docs.kicad.org/>.
- [5] Nordic Semiconductor, “Bluetooth Low Energy Fundamentals.”. Available: <https://academy.nordicsemi.com/>.
- [6] Seeed Studio, “Seeed Studio XIAO nRF52840 Sense Datasheet.”. Available: <https://files.seeedstudio.com/wiki/XIAO/Seeed-Studio-XIAO-Series-SOM-Datasheet.pdf>.
- [7] Analog Devices, “DS3231 Extremely Accurate I<sup>2</sup>C-Integrated RTC Datasheet.”. Available: <https://www.analog.com/media/en/technical-documentation/data-sheets/ds3231.pdf>.
- [8] Espressif Systems, “ESP32-WROOM-32 Datasheet.”. Available: [https://documentation.espressif.com/esp32\\_datasheet\\_en.pdf](https://documentation.espressif.com/esp32_datasheet_en.pdf).
- [9] SparkFun Electronics, “Nokia 5110 (PCD8544) LCD Datasheet.”. Available: <https://www.sparkfun.com/datasheets/LCD/Monochrome/Nokia5110.pdf>.
- [10] Vishay, “16×2 Character LCD Datasheet.”. Available: <https://www.vishay.com/docs/37484/lcd016n002bcfh.pdf>.
- [11] Arnab Roy, “Motion Data Recorder Project Repository,” GitHub Repository.. Available: [https://github.com/ar0y1011/2026\\_GIPDEI\\_ARNAB\\_ROY.git](https://github.com/ar0y1011/2026_GIPDEI_ARNAB_ROY.git).
- [12] Chandan, “Motion Data Recorder Project Files,” Google Drive Repository.. Available: <https://drive.google.com/drive/folders/1FidejkAdgzLm0jK5utsCs0yzdaDB158G?usp=sharing>.
- [13] OpenAI, “ChatGPT.”. Available: <https://chatgpt.com/>.
- [14] GitHub, “GitHub Copilot.”. Available: <https://github.com/features/copilot>.